UNIVERSIDAD PERUANA
CAYETANO HEREDIA

ManiFlex - Entregable 10

EQUIPO 08

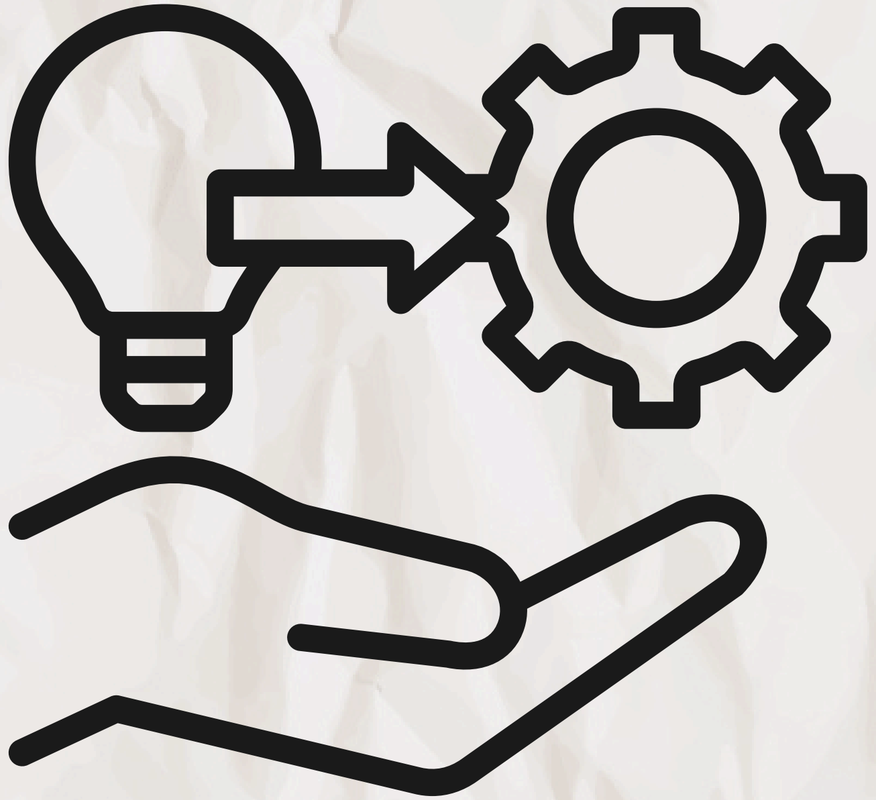
- **HOYOS PUENTE, ABNER MIHAILS** (DISEÑADOR)
- **LAZARO ABANTO, DIEGO DAVID** (LÍDER DEL EQUIPO)
- **MARTÍNEZ AQUINO, VALERIA JIMENA** (PROGRAMADORA)
- **MUÑOZ HUATUCO, KIARA PIERINA** (ENCARGADA DE LA ELECTRÓNICA)
- **NEYRA QUEZADA, GABRIELA ESTEFANÍA** (ENCARGADA DEL MODELADO)
- **RAMOS GUERRA, ALEJANDRO JOSUE** (RESPONSABLE DE INVESTIGACIÓN)



EQUIPO 08 - FUNDAMENTOS DE BIODISEÑO

CASO CLINICO 06 - TEC
2026 - II

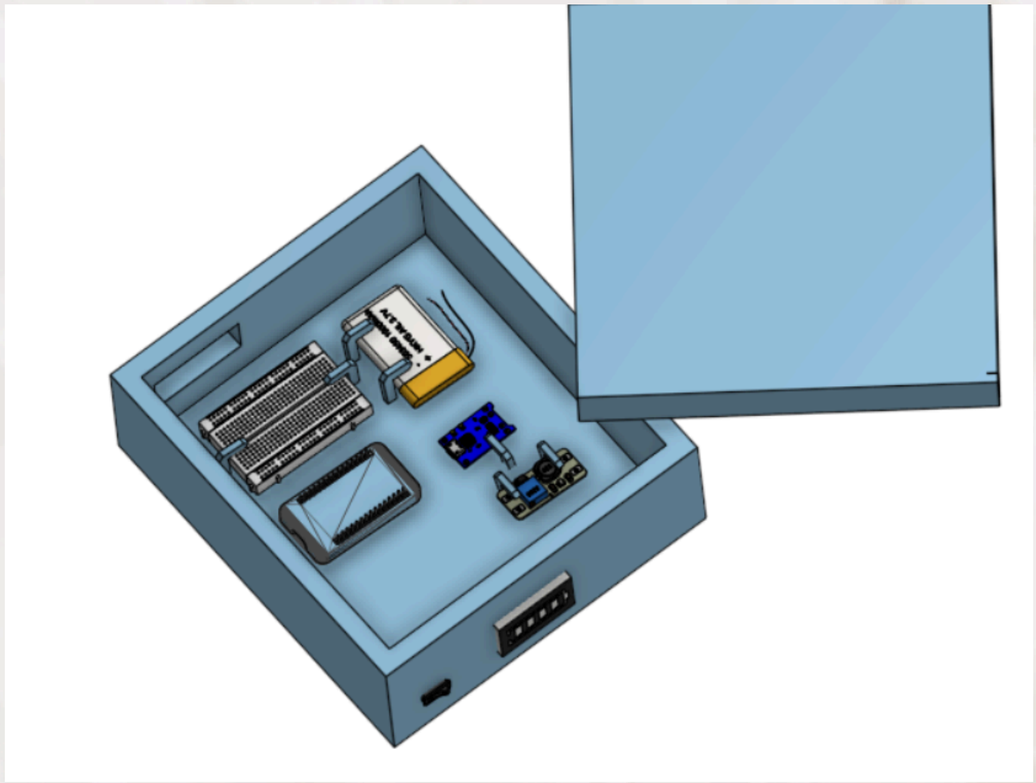
IMPLEMENTACIÓN PRELIMINAR DE LOS SUBSISTEMAS

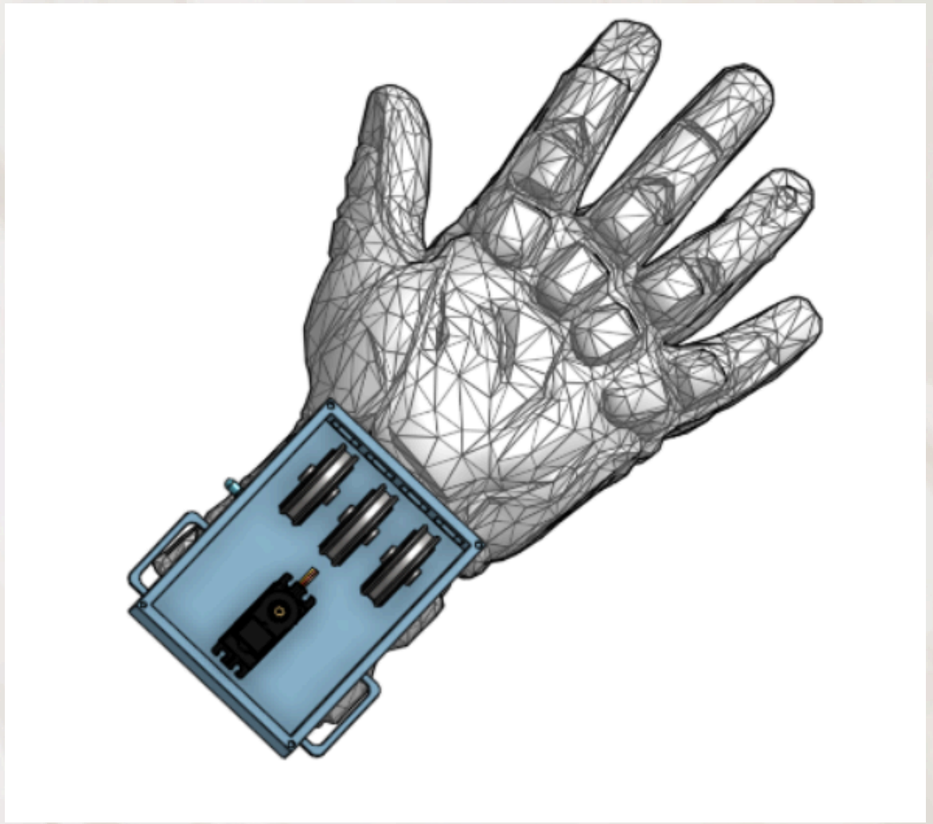


Mecánica y fabricación

Implementación esperada: Piezas fabricadas de acuerdo con los modelos CAD y la especificaciones del diseño.

Evidencia:

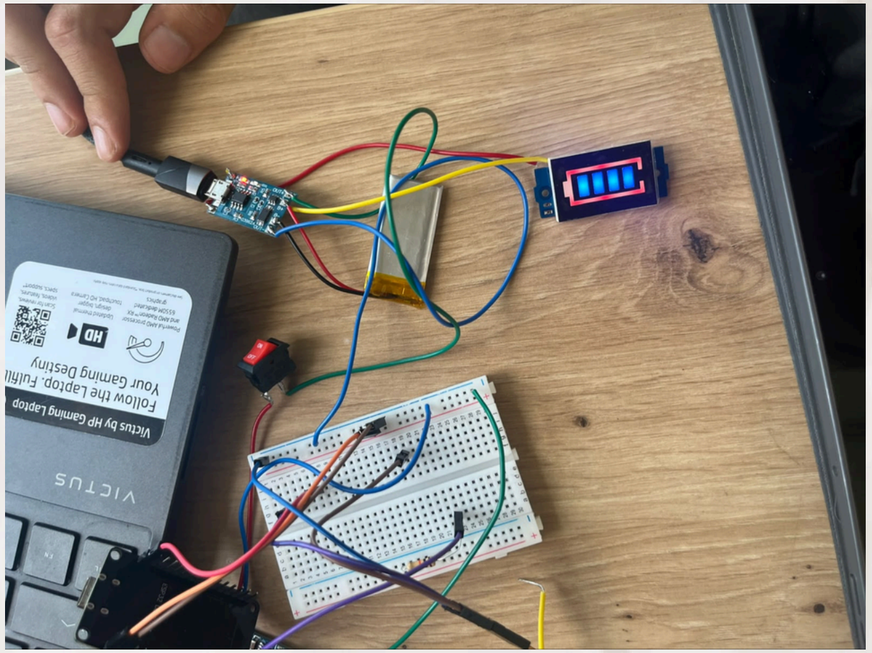


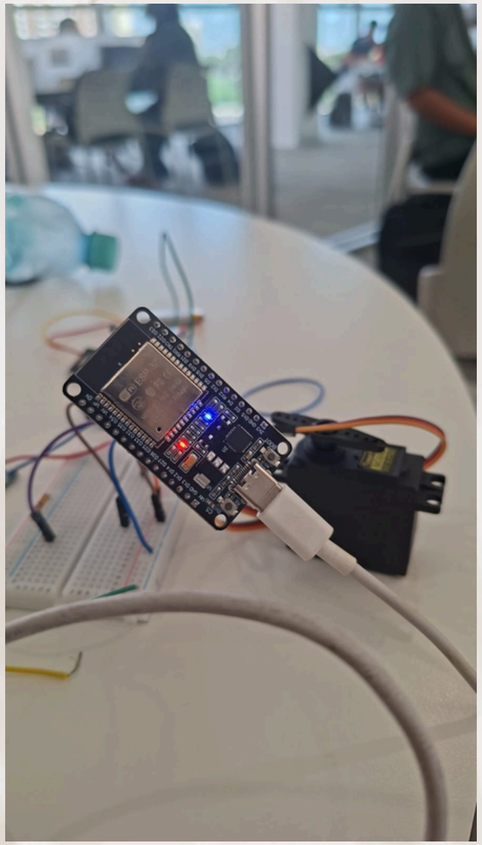


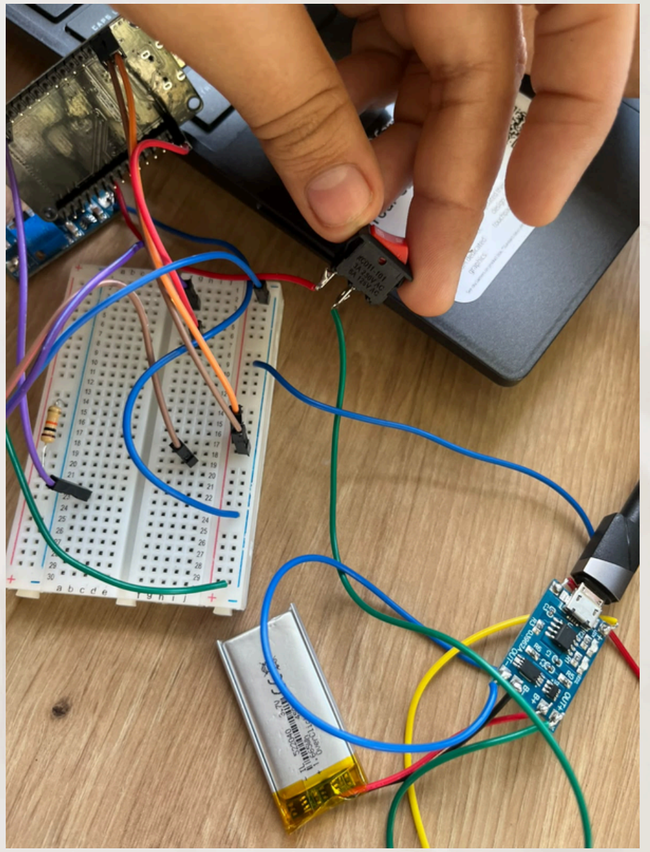
Electrónica y sensores

Tipo de verificación: Verificación dimensional y funcional de las piezas fabricadas.

Evidencia:



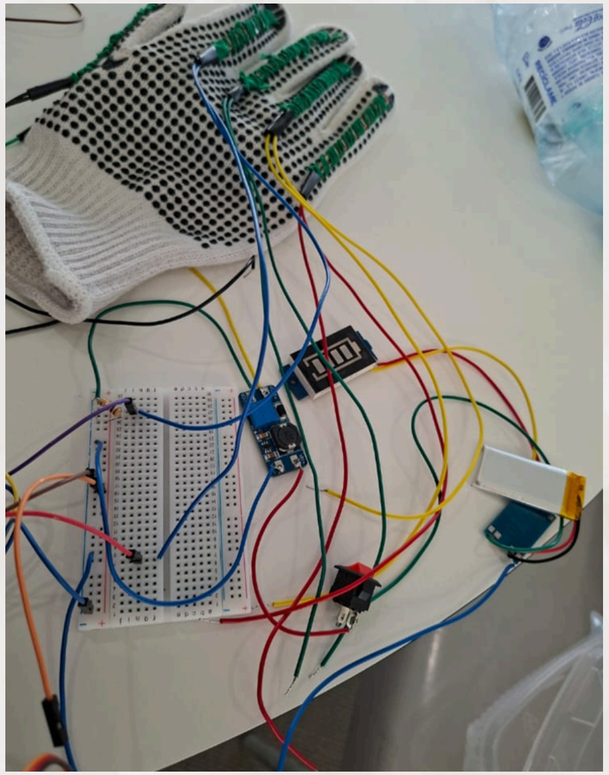


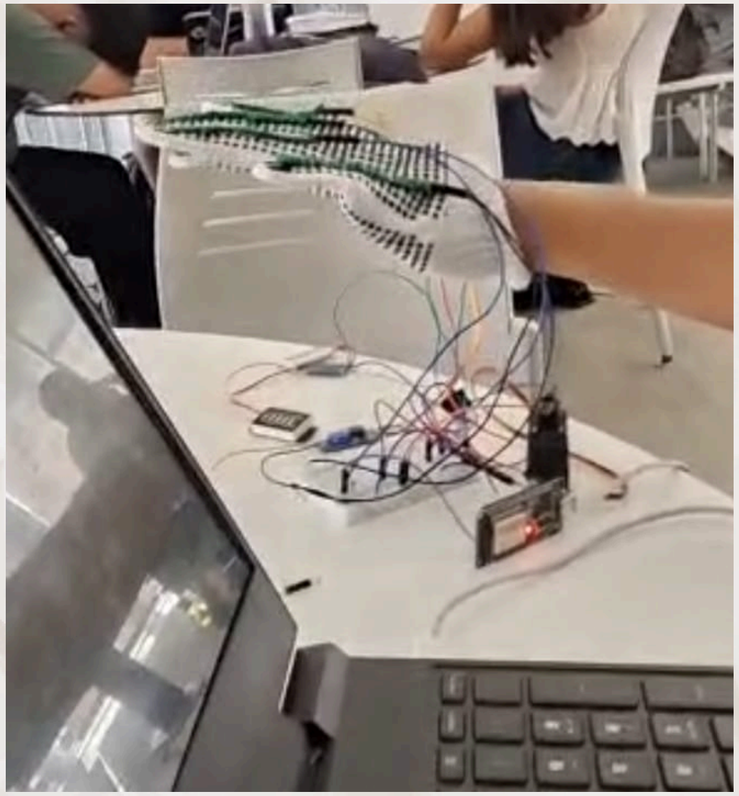


Electrónica y sensores

Tipo de verificación: Verificación de conexiones eléctricas y funcionamiento individual de sensores y actuadores.

Evidencia:





Prueba de funcionamiento

```
ctura:399 baseline:430 caida:31 angulo:25
ctura:398 baseline:430 caida:32 angulo:22
ctura:398 baseline:430 caida:32 angulo:19
ctura:399 baseline:430 caida:31 angulo:16
ctura:399 baseline:430 caida:31 angulo:13
ctura:399 baseline:430 caida:31 angulo:10
ctura:398 baseline:430 caida:32 angulo:7
```

Electrónica y sensores

Tipo de verificación: Verificación de conexiones eléctricas y funcionamiento individual de sensores y actuadores.

Conexiones:

N.°	Origen	Destino
1. Etapa de alimentación		
1	Batería LiPo 3.7 V (+)	B+ del módulo TP4056
2	Batería LiPo 3.7 V (–)	B– del módulo TP4056
3	OUT+ del TP4056	Terminal del interruptor (rocker switch)
4	Terminal del interruptor	VIN+ del MT3608
5	OUT– del TP4056	Riel GND
6	VIN– del MT3608	Riel GND
7	VOUT– del MT3608	Riel GND
8	VOUT+ del MT3608 (ajustado a 5.0 V)	Riel +5 V
2. Microcontrolador ESP32		
1	VIN	Riel +5 V
2	GND	Riel GND
3	3V3	Terminal 1 del sensor flex
4	D34 (ADC1, entrada analógica)	Nodo del divisor de tensión
5	D13 (salida PWM)	Cable de señal del servomotor
3. Sensor flex y divisor de tensión		
1	Sensor flex, terminal 1	Pin 3V3 del ESP32
2	Sensor flex, terminal 2	Nodo del divisor (común con D34 y la resistencia)
3	Resistencia de 10 kΩ, pata 1	Nodo del divisor
4	Resistencia de 10 kΩ, pata 2	Riel GND
	$V_{D34} = 3.3\ V \cdot 10\ k\Omega / (R_{flex} + 10\ k\Omega)$. Al flexionar el sensor aumenta R_{flex}	
4. Servomotor MG995		
1	Rojo (+)	Riel +5 V
2	Marrón o negro (GND)	Riel GND
3	Naranja o amarillo (señal)	Pin D13 del ESP32

Programación y control

Implementación esperada: Código de programación desarrollado de acuerdo con el diagrama de flujo de control.

Módulo 1 · Configuración

- PIN_FLEX = 34: entrada analógica (ADC1) donde se lee el divisor del flex.
- PIN_SERVO = 13: salida PWM hacia el MG995.
- Servo: ANGULO_REPOSO = 0° (hilo flojo) y ANGULO_X = 90° (tracción).
- Sensor: 16 muestras por promedio, UMBRAL_ON = 80 y UMBRAL_OFF = 40.
- Todo en constantes al inicio para ajustar sin tocar la lógica.

```
const int PIN_FLEX  = 34;
const int PIN_SERVO = 13;

const int ANGULO_REPOSO = 0;
const int ANGULO_X      = 90;
const unsigned long
  INTERVALO_PASO_MS = 25;

const int MUESTRAS   = 16;
const int UMBRAL_ON  = 80;
const int UMBRAL_OFF = 40;
```

Programación y control

Implementación esperada: Código de programación desarrollado de acuerdo con el diagrama de flujo de control.

Módulo 2 - Lectura Flex

- leerFlex() devuelve el promedio de 16 lecturas del ADC.
- Promediar reduce el ruido, que en el ESP32 es alto.
- El ADC trabaja a 12 bits (0 a 4095) con atenuación de 11 dB (hasta ~3.3 V).
- Divisor: 3V3, flex, D34 y 10 kΩ a GND. Al doblar el flex, el voltaje baja.

```
int leerFlex() {
  long suma = 0;
  for (int i = 0; i < MUESTRAS; i++) {
    suma += analogRead(PIN_FLEX);
    delayMicroseconds(200);
  }
  return (int)(suma / MUESTRAS);
}
```

Programación y control

Implementación esperada: Código de programación desarrollado de acuerdo con el diagrama de flujo de control.

Módulo 3 - Arranque: setup()

- Espera de 3 s para ponerse el guante (LED parpadeo lento).
- Durante 2 s toma la referencia (baseline) con la mano relajada (LED rápido).
- Recién después activa el servo, para no sumar su pico de corriente al arranque del ESP32.
- El servo parte en ANGULO_REPOSO.

```
parpadear(3000, 1000);   // espera

// referencia, mano relajada
while (millis() - inicio < 2000) {
  suma += leerFlex(); n++;
}
baseline = suma / n;

// recién ahora el servo
servo.attach(PIN_SERVO, 500, 2400);
servo.write(ANGULO_REPOSO);
```

Programación y control

Implementación esperada: Código de programación desarrollado de acuerdo con el diagrama de flujo de control.

Módulo 4- Detección con histéresis

- caida = baseline − lectura: cuánto se flexionó el dedo respecto al reposo.
- Si caida ≥ UMBRAL_ON (80), se activa la tracción.
- Si caida ≤ UMBRAL_OFF (40), se suelta el hilo.
- Entre 40 y 80 mantiene el estado: evita que el servo vibre por ruido.

```
int lectura = leerFlex();
int caida = baseline - lectura;

if (!traccionActiva && caida >= UMBRAL_ON) {
  traccionActiva = true;
  anguloObjetivo = ANGULO_X;
} else if (traccionActiva &&
           caida <= UMBRAL_OFF) {
  traccionActiva = false;
  anguloObjetivo = ANGULO_REPOSO;
}
```

Programación y control

Implementación esperada: Código de programación desarrollado de acuerdo con el diagrama de flujo de control.

Módulo 5- Movimiento suave del servo

- El servo no salta: avanza 1° cada 25 ms hacia el ángulo objetivo.
- De 0° a 90° tarda unos 2.3 s, lo que protege al paciente y a la fuente.
- El LED integrado queda encendido fijo mientras hay tracción.
- Se ajusta con PASO_GRADOS e INTERVALO_PASO_MS.

```
digitalWrite(PIN_LED,
  traccionActiva ? HIGH : LOW);

if (millis() - ultimoPaso >= 25) {
  ultimoPaso = millis();
  if (anguloActual < anguloObjetivo) {
    anguloActual = min(
      anguloActual + PASO_GRADOS,
      anguloObjetivo);
    servo.write(anguloActual);
  }
}
```

Programación y control

Implementación esperada: Código de programación desarrollado de acuerdo con el diagrama de flujo de control.

Módulo 6- Monitor Serie y ajustes

- Con USB, el Monitor Serie (115200) muestra lectura, baseline, caída y ángulo.
- UMBRAL_ON: más bajo = más sensible (no menos de ~30, por el ruido).
- ANGULO_X: sale de la geometría del hilo (hilo a jalar ÷ radio del brazo del servo).
- INTERVALO_PASO_MS: velocidad del movimiento.

```
lectura:429 baseline:430 caida:1  angulo:0
lectura:398 baseline:430 caida:32 angulo:7

// Ajustes principales
UMBRAL_ON  = 80
UMBRAL_OFF = 40
ANGULO_X   = 90
INTERVALO_PASO_MS = 25
```

Gracias